

SISTEMA DE AUTENTICACIÓN HÍBRIDO CON NODE.JS

Documentación Técnica — Backend 2 | Proyecto Final

Proyecto: ChronoShop Backend

Alumno: Bautista Cortez Pincha

Repositorio: github.com/bautistacorte05/chronoshop-backend

Fecha: 2026-05-20

Implementación de un sistema de autenticación híbrido que combina Passport Local (email + bcrypt), OAuth 2.0 (GitHub y Google), tokens JWT en cookie httpOnly y sesiones persistentes en MongoDB.

Documentación Técnica — Backend 2

Proyecto: ChronoShop Backend **Alumno:** Bautista Cortez Pincha **Repositorio:** <https://github.com/bautistacorte05/chronoshop-backend> **Fecha:** 2026-05-20

1. Presentación del Proyecto

1.1 Descripción general

ChronoShop es un e-commerce de relojes construido con Node.js, Express y MongoDB. A partir del proyecto de Backend 1 (que implementaba gestión de productos y carritos con arquitectura por capas DAO/Manager), en esta entrega se extiende el sistema agregando un **sistema de autenticación híbrido** que combina tres mecanismos:

- **Autenticación Local:** registro y login con email/contraseña usando bcrypt para hashing y Passport.js como middleware.
- **OAuth 2.0:** login social mediante GitHub y Google usando estrategias de Passport.
- **JWT + Sesiones coexistentes:** al autenticarse, el servidor emite simultáneamente un **token JWT** (en body y en cookie `authToken` httpOnly) y crea una **sesión** en MongoDB (via connect-mongo). Ambos mecanismos conviven.

1.2 Objetivo arquitectónico

Demostrar que JWT y sesiones no son mutuamente excluyentes. Las rutas de la API REST usan el JWT (stateless, escalable), mientras que las vistas SSR (Handlebars) usan la sesión de Passport (stateful, con persistencia en MongoDB). El resultado es un sistema donde un cliente browser navega con sesión y un cliente API consume con Bearer token.

1.3 Estrategias de autenticación implementadas

Estrategia	Mecanismo	Librería
Local	Email + contraseña (bcrypt)	passport-local
GitHub OAuth	OAuth 2.0 con código de autorización	passport-github2
Google OAuth	OAuth 2.0 con OpenID Connect	passport-google-oauth2

1.4 Justificación del enfoque

Se eligió el enfoque híbrido porque ChronoShop ya tenía dos capas diferenciadas desde Backend 1: rutas API (`/api/*`) y rutas de vistas (`/products`, `/carts`). Cada canal se beneficia de un mecanismo distinto. El JWT en cookie httpOnly mitiga XSS (el script no puede leer la cookie), y `sameSite: 'Lax'` mitiga CSRF sin romper los redirects de OAuth.

2. Arquitectura del Proyecto

2.1 Estructura de carpetas completa

chronoshop-backend/	
■■■ app.js	← Entry point: Express + middlewares globales
■■■ seed.js	← Script de carga inicial de productos
■■■ package.json	
■■■ .env.example	
■■■ vitest.config.js	
■	
■■■ src/	
■■■ config/	
■ ■■■ passport.config.js	← configurePassport(userManager): registra las 3 estrategias
■	
■■■ models/	
■ ■■■ Product.js	← Esquema Mongoose de producto (existente)
■ ■■■ Cart.js	← Esquema Mongoose de carrito (existente)
■ ■■■ User.js	← NUEVO: email, password, role, provider, githubId, googleId
■	
■■■ dao/	
■ ■■■ mongo/	
■ ■■■ ProductMongoDAO.js	← (existente)
■ ■■■ CartMongoDAO.js	← (existente)
■ ■■■ UserMongoDAO.js	← NUEVO: create, findByEmail, findById, findByGithubId, find...
■	
■■■ managers/	
■ ■■■ ProductManager.js	← (existente)
■ ■■■ CartManager.js	← (existente)
■ ■■■ UserManager.js	← NUEVO: register (bcrypt), findOrCreateGithub, findOrCreate...
■	
■■■ strategies/	
■ ■■■ local.strategy.js	← NUEVO: makeLocalStrategy(userManager)
■ ■■■ github.strategy.js	← NUEVO: makeGithubStrategy(userManager)
■ ■■■ google.strategy.js	← NUEVO: makeGoogleStrategy(userManager)
■	
■■■ controllers/	
■ ■■■ auth.controller.js	← NUEVO: makeAuthController(userManager) con 6 handlers
■	
■■■ middlewares/	
■ ■■■ authJWT.js	← NUEVO: verifica cookie authToken o Bearer header
■ ■■■ authRole.js	← NUEVO: authAdmin - 401 sin user, 403 sin rol
■	
■■■ routes/	
■ ■■■ api/	
■ ■■■ products.router.js	← Modificado: POST/PUT/DELETE protegidos con JWT + admin
■ ■■■ carts.router.js	← (existente)
■ ■■■ auth.router.js	← NUEVO: /api/v1/auth/*, /api/v1/session, /profile, /admin
■ ■■■ views/	
■ ■■■ products.router.js	← (existente)
■ ■■■ carts.router.js	← (existente)
■ ■■■ auth.router.js	← NUEVO: /login, /register, /auth/github, /auth/google
■	
■■■ views/	
■ ■■■ layouts/main.handlebars	← (existente)
■ ■■■ products.handlebars	← (existente)
■ ■■■ cart.handlebars	← (existente)
■ ■■■ login.handlebars	← NUEVO
■ ■■■ register.handlebars	← NUEVO
■	
■■■ public/	← (existente)

2.2 Explicación de cada capa

`config/` Centraliza la configuración de Passport. `passport.config.js` exporta `configurePassport(userManager)` que registra `serializeUser`, `deserializeUser` y las tres estrategias. Se ejecuta una única vez al iniciar la app, antes de montar las rutas.

`models/` Esquemas Mongoose. `User.js` define los campos del usuario con soporte para autenticación local y OAuth. El campo `provider` indica el origen del registro (`local`, `github`, `google`). Los campos `githubId` y `googleId` son nulos para usuarios locales.

dao/ Objetos de acceso a datos: operaciones de base de datos atómicas sin lógica de negocio. **UserMongoDAO** ofrece cinco métodos de consulta. No sabe nada de bcrypt ni de validaciones.

managers/ Capa de lógica de negocio. **UserManager** orquesta registro (verifica duplicados, hashea contraseña, persiste), búsquedas por distintos identificadores y el patrón **findOrCreate** para OAuth.

strategies/ Una estrategia de Passport por archivo, implementadas como factories que reciben **userManager** como dependencia. Esto las hace testeables independientemente de la base de datos.

`controllers/` Funciones de manejo de request/response. `auth.controller.js` exporta `makeAuthController(userManager)` que retorna un objeto con los seis handlers. No accede a la base de datos directamente — delega a `userManager`.

middlewares/ Funciones puras encadenables antes de los controllers. **authJWT** verifica el token JWT (sin DB). **authAdmin** verifica el rol (sin DB). El orden importa: **authJWT** siempre va antes de **authAdmin**.

routes/ Solo mapeo de URLs a middleware chains y controllers. No contiene lógica de negocio.

2.3 Diagrama del flujo de autenticación

[illegible]

```

    ■
    authJWT middleware:
    ■■■ token = req.cookies.authToken || Bearer header
    ■■■ jwt.verify(token, JWT_SECRET) ■■■ payload { userId, role }
    ■■■ req.user = payload
    ■■■ next()
    ■
    authAdmin middleware (solo /admin):
    ■■■ req.user.role !== 'admin' ■■■ 403
    ■■■ next()
    ■
    controller ■■■ res.json(data)

```

3. Implementación Técnica

3.1 Registro de Usuario

Endpoint: **POST** **/api/v1/auth/register**

Modelo User (src/models/User.js)

```

import mongoose from 'mongoose'

const userSchema = new mongoose.Schema({
  firstName: { type: String, trim: true },
  lastName: { type: String, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, default: null }, // null para usuarios OAuth
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  provider: { type: String, enum: ['local', 'github', 'google'], default: 'local' },
  githubId: { type: String, default: null },
  googleId: { type: String, default: null },
  avatar: { type: String, default: null },
  createdAt: { type: Date, default: Date.now }
})

export default mongoose.model('User', userSchema)

```

Hash de contraseña con bcrypt (src/managers/UserManager.js)

```

async register({ firstName, lastName, email, password }) {
  const exists = await this.dao.findByEmail(email)
  if (exists) throw new Error('EMAIL_TAKEN')
  const rounds = parseInt(process.env.BCRYPT_ROUNDS) || 10
  const hash = await bcrypt.hash(password, rounds)
  return this.dao.create({ firstName, lastName, email, password: hash, provider: 'local' })
}

```

El texto plano **nunca se almacena**. **bcrypt.hash** genera un hash con 10 salt rounds. El hash resultante tiene el formato **\$2b\$10\$...** y es irreversible.

Validación de duplicados (src/controllers/auth.controller.js)

```

register: async (req, res) => {
  try {

```

```
const { firstName, lastName, email, password } = req.body
if (!email || !password)
  return res.status(400).json({ error: 'Email y contraseña son requeridos' })
if (password.length < 6)
  return res.status(400).json({ error: 'La contraseña debe tener al menos 6 caracteres' })
const user = await userManager.register({ firstName, lastName, email, password })
res.status(201).json({
  message: 'Usuario registrado correctamente',
  user: { _id: user._id, email: user.email, role: user.role }
})
} catch (err) {
  if (err.message === 'EMAIL_TAKEN')
    return res.status(409).json({ error: 'El email ya está registrado' })
  res.status(500).json({ error: 'Error interno del servidor' })
}
}
```

Request y Response reales

Request:

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "firstName": "María",
  "lastName": "García",
  "email": "maria@chronoshop.com",
  "password": "secure123"
}
```

Response 201 — Registro exitoso:

```
{
  "message": "Usuario registrado correctamente",
  "user": {
    "_id": "6a0db79c6cfd596feab7b13d",
    "email": "maria@chronoshop.com",
    "role": "user"
  }
}
```

Response 409 — Email duplicado:

```
{
  "error": "El email ya está registrado"
}
```

3.2 Login Local (Passport LocalStrategy)

Endpoint: `POST /api/v1/auth/login`

Configuración de Passport LocalStrategy (src/strategies/local.strategy.js)

```
import { Strategy as LocalStrategy } from 'passport-local'
import bcrypt from 'bcrypt'

export const makeLocalStrategy = (userManager) =>
```

```

new LocalStrategy(
  { usernameField: 'email' },
  async (email, password, done) => {
    try {
      const user = await userManager.findByEmail(email)
      if (!user || user.provider !== 'local') return done(null, false)
      const match = await bcrypt.compare(password, user.password)
      if (!match) return done(null, false)
      done(null, user)
    } catch (err) {
      done(err)
    }
  }
)

```

`usernameField: 'email'` configura que Passport lea `req.body.email` en lugar del campo por defecto `username`. La estrategia rechaza usuarios OAuth (`user.provider !== 'local'`) para evitar que intenten loguear con una contraseña que nunca establecieron.

Generación del JWT (`src/controllers/auth.controller.js`)

```

const generateToken = (user) =>
  jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRATION || '1h' }
  )

```

El payload contiene `userId` y `role`. La expiración se configura en `.env` como `JWT_EXPIRATION=1h`.

Envío del token: *body + cookie* (`src/controllers/auth.controller.js`)

```

loginLocal: (req, res) => {
  const token = generateToken(req.user)
  // En cookie httpOnly (protegida contra XSS)
  res.cookie('authToken', token, {
    httpOnly: true,
    sameSite: 'Lax',
    secure: process.env.NODE_ENV === 'production',
    maxAge: 3600000
  })
  // También en body (para clientes API como Postman)
  res.json({
    message: 'Login exitoso',
    token,
    user: { _id: req.user._id, email: req.user.email, role: req.user.role }
  })
}

```

El token viaja dos veces: 1. **En el body JSON:** para clientes API, móviles, Postman. 2. **En la cookie `authToken`:** para el browser, de forma automática en cada request.

Request y Response reales

Request:

```

POST /api/v1/auth/login
Content-Type: application/json

```

Response 200 — Login exitoso:

Header Set-Cookie generado:

JWT Payload decodificado (sección central del token, base64):

3.3 Login OAuth (GitHub y Google)

Endpoints: GET /auth/github, GET /auth/google

Configuración de la estrategia GitHub (src/strategies/github.strategy.js)

Sistema de Autenticación Híbrido con Node.js


```
}
)
```

Creación de usuario si no existe (src/managers/UserManager.js)

```
async findOrCreateGithub({ githubId, firstName, email, avatar }) {
  let user = await this.dao.findByGithubId(githubId)
  if (!user) {
    user = await this.dao.create({
      githubId, firstName, email, avatar, provider: 'github'
    })
  }
  return user
}
```

La operación es **idempotente**: si el usuario ya existe (login previo con GitHub), se retorna el existente. Si es la primera vez, se crea con **provider: 'github'** y sin password.

Cómo se mantiene la sesión en OAuth

```
// src/routes/views/auth.router.js
router.get('/auth/github/callback',
  passport.authenticate('github', { failureRedirect: '/login', session: true }),
  ctrl.oauthCallback // emite JWT + cookie, luego redirect /products
)

// src/controllers/auth.controller.js
oauthCallback: (req, res) => {
  const token = generateToken(req.user) // req.user viene de Passport
  setAuthCookie(res, token)
  res.redirect('/products')
}
```

1. Passport llama **req.login(user)** automáticamente (por **session: true**), creando la sesión en MongoDB. 2. El callback **oauthCallback** emite adicionalmente el JWT en cookie. 3. El browser recibe las dos cookies (**authToken** y **connect.sid**) y es redirigido a **/products**.

Fragmentos de configuración en passport.config.js

```
export const configurePassport = (userManager) => {
  passport.serializeUser((user, done) => done(null, user._id.toString()))
  passport.deserializeUser(async (id, done) => {
    try {
      const user = await userManager.findById(id)
      done(null, user)
    } catch (err) {
      done(err)
    }
  })
}

passport.use(makeLocalStrategy(userManager))
// Solo registra OAuth si las credenciales están configuradas
if (process.env.GITHUB_CLIENT_ID && process.env.GITHUB_CLIENT_SECRET)
  passport.use(makeGithubStrategy(userManager))
if (process.env.GOOGLE_CLIENT_ID && process.env.GOOGLE_CLIENT_SECRET)
  passport.use(makeGoogleStrategy(userManager))
}
```

La comprobación condicional evita que el servidor crashee si las credenciales OAuth no están configuradas (útil en entornos de desarrollo sin app registrada).

3.4 Sistema de Sesiones

Configuración de express-session + connect-mongo (app.js)

```
import session from 'express-session'
import MongoStore from 'connect-mongo'

app.use(cookieParser())
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  store: MongoStore.create({ mongoUrl: process.env.MONGODB_URI }),
  cookie: {
    maxAge: 3600000,
    httpOnly: true,
    sameSite: 'Lax',
    secure: process.env.NODE_ENV === 'production'
  }
}))
app.use(passport.initialize())
app.use(passport.session())
```

resave: false y **saveUninitialized: false** evitan escribir sesiones vacías en MongoDB. El store persiste las sesiones en la colección **sessions** de la misma base de datos.

Ejemplo de documento de sesión en MongoDB

```
{
  "_id": "FiVdATBszatnEkVV7Q1-XFNeouGSE8Je",
  "expires": { "$date": "2026-05-20T11:37:56.000Z" },
  "session": {
    "cookie": {
      "originalMaxAge": 3600000,
      "expires": "2026-05-20T11:37:56.141Z",
      "httpOnly": true,
      "sameSite": "Lax",
      "secure": false,
      "path": "/"
    },
    "passport": {
      "user": "6a0db79c6cfd596feab7b13d"
    }
  }
}
```

Solo se persiste el **_id** del usuario (serializado por **passport.serializeUser**). En cada request con session cookie válida, **deserializeUser** reconstruye el objeto completo desde MongoDB.

Endpoint GET /api/v1/session

```
getSession: (req, res) => {
  if (!req.user) return res.status(401).json({ error: 'Sin sesión activa' })
```

```

    res.json({
      sessionId: req.sessionID,
      user: { _id: req.user._id, email: req.user.email, role: req.user.role }
    })
  }
}

```

Response real — sesión activa:

```

{
  "sessionId": "FiVdATBszatnEkVV7Q1-XFNeouGSE8Je",
  "user": {
    "_id": "6a0db79c6cfd596feab7b13d",
    "email": "maria@chronoshop.com",
    "role": "user"
  }
}

```

3.5 Rutas Protegidas

GET /api/v1/profile — protegida por JWT

Middleware authJWT (src/middlewares/authJWT.js):

```

import jwt from 'jsonwebtoken'

export const authJWT = (req, res, next) => {
  const bearerToken = req.headers.authorization?.startsWith('Bearer ')
    ? req.headers.authorization.split(' ')[1]
    : undefined
  const token = req.cookies?.authToken || bearerToken
  if (!token) return res.status(401).json({ error: 'No autenticado' })
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET)
    req.user = payload // { userId, role, iat, exp }
    next()
  } catch {
    res.status(401).json({ error: 'Token inválido o expirado' })
  }
}

```

Response real — con token válido (200):

```

{
  "user": {
    "_id": "6a0db79c6cfd596feab7b13d",
    "email": "maria@chronoshop.com",
    "role": "user",
    "firstName": "María",
    "lastName": "García"
  }
}

```

Response real — sin token (401):

```

{
  "error": "No autenticado"
}

```

GET /api/v1/admin — protegida por JWT + rol

Middleware authAdmin (src/middlewares/authRole.js):

```
export const authAdmin = (req, res, next) => {
  if (!req.user) return res.status(401).json({ error: 'No autenticado' })
  if (req.user.role !== 'admin') return res.status(403).json({ error: 'Acceso denegado' })
  next()
}
```

Montaje de la ruta:

```
router.get('/admin', authJWT, authAdmin, ctrl.getAdmin)
```

Response real — usuario con role: 'user' (403):

```
{
  "error": "Acceso denegado"
}
```

Tabla de respuestas de error de autenticación:

Caso	Status	Body
Sin token	401	{ "error": "No autenticado" }
Token expirado o inválido	401	{ "error": "Token inválido o expirado" }
Token válido, rol insuficiente	403	{ "error": "Acceso denegado" }

Rutas y su protección:

Ruta	Método	Protección
GET /api/products	GET	Pública
POST /api/products	POST	authJWT + authAdmin
PUT /api/products/:pid	PUT	authJWT + authAdmin
DELETE /api/products/:pid	DELETE	authJWT + authAdmin
GET /api/v1/session	GET	Session (Passport)
GET /api/v1/profile	GET	authJWT
GET /api/v1/admin	GET	authJWT + authAdmin

Response real — POST /api/products sin token (401):

```
{
  "error": "No autenticado"
}
```

3.6 Logout

Endpoint: POST /api/v1/auth/logout

```
logout: (req, res) => {
```

```
req.logout((err) => {
  if (err) return res.status(500).json({ error: 'Error al cerrar sesión' })
  req.session.destroy((destroyErr) => {
    res.clearCookie('authToken')
    res.clearCookie('connect.sid')
    res.json({ message: 'Logout exitoso' })
  })
})
}
```

Secuencia de destrucción: 1. `req.logout()` — Passport limpia `req.user` y elimina la referencia de la sesión. 2. `req.session.destroy()` — Elimina el documento de sesión de la colección `sessions` en MongoDB. 3. `res.clearCookie('authToken')` — El browser elimina la cookie JWT. 4. `res.clearCookie('connect.sid')` — El browser elimina la cookie de sesión.

> **Nota sobre el JWT:** Al ser stateless, el servidor no puede revocar un JWT ya emitido. El cliente browser lo pierde al eliminarse la cookie. Clientes que guardaron el token en memoria deben descartarlo ellos mismos. La expiración corta (1h) limita la ventana de exposición post-logout.

Response real — Logout exitoso:

```
{
  "message": "Logout exitoso"
}
```

Response real — GET /api/v1/session después del logout (401):

```
{
  "error": "Sin sesión activa"
}
```

4. Seguridad y Decisiones Arquitectónicas

¿Dónde vive el rol y por qué?

El rol vive en **dos lugares simultáneamente**:

1. **Base de datos** (`User.role`): fuente de verdad persistente, se actualiza cuando cambia. 2. **JWT payload** (`{ userId, role }`): copia inmutable hasta la expiración del token.

El middleware `authJWT` lee el rol desde el payload del JWT **sin consultar la base de datos**, haciendo la autorización O(1). La contrapartida es el siguiente punto.

¿Qué ocurre si el rol cambia con un token ya emitido?

Si se degrada un admin a usuario común, su JWT sigue teniendo `role: 'admin'` hasta expirar (máximo 1h). Durante ese período podría acceder a rutas de admin. Las mitigaciones aplicadas:

- **Expiración corta (1h):** limita la ventana de riesgo sin infraestructura adicional.
- **Solución completa** (no implementada por complejidad): mantener una blacklist de tokens invalidados en Redis/MongoDB.

La expiración de 1h es el balance elegido para este proyecto entre seguridad y rendimiento.

¿Por qué cookie + JWT y no solo uno?

Solo cookie de sesión	Solo JWT en localStorage	JWT en cookie httpOnly
Depende del servidor para cada request	Vulnerable a XSS (scripts pueden robarlo)	Protegido contra XSS (scripts no pueden leer cookies httpOnly)
No escala sin session store compartido	Escala bien	Escala bien con expiración
Logout inmediato	Logout requiere blacklist	Logout limpia cookie

La combinación elegida da lo mejor de ambos mundos: el JWT viaja en una cookie que el browser gestiona automáticamente y el script nunca puede leer.

¿Cómo se mitigó CSRF?

Con `sameSite: 'Lax'` en la cookie `authToken`:

- `Strict` bloquearía los redirects de OAuth (GitHub/Google redirigen a nuestro callback), rompiendo el flujo de login social.
- `Lax` permite GET cross-site (necesario para OAuth callbacks) pero bloquea cookies en requests cross-site iniciados por `<form>`, `fetch` de terceros o `XMLHttpRequest`.
- Para mayor protección en producción se puede agregar `csrf` middleware.

¿Cómo difiere entorno local vs. producción?

Configuración	Local	Producción
<code>cookie.secure</code>	<code>false</code> (HTTP)	<code>true</code> (solo HTTPS)
<code>NODE_ENV</code>	<code>development</code>	<code>production</code>
<code>MONGODB_URI</code>	<code>localhost:27017</code>	Atlas o instancia cloud
OAuth callbacks	<code>http://localhost:8080/...</code>	<code>https://dominio.com/...</code>

La condición `secure: process.env.NODE_ENV === 'production'` en ambas cookies maneja la diferencia sin cambios de código.

5. Evidencia Real de Funcionamiento

Todos los ejemplos siguientes son salidas reales del servidor corriendo en `http://localhost:8080`.

5.1 Registro exitoso

POST `/api/v1/auth/register`

```
{
  "message": "Usuario registrado correctamente",
  "user": {
    "_id": "6a0db79c6cfd596feab7b13d",
    "email": "maria@chronoshop.com",
    "role": "user"
  }
}
```

} }

```
}  
}
```

5.7 GET /api/v1/profile — ruta protegida por JWT

```
GET /api/v1/profile  
Authorization: Bearer eyJhbGci...
```

```
{  
  "user": {  
    "_id": "6a0db79c6cfd596feab7b13d",  
    "email": "maria@chronoshop.com",  
    "role": "user",  
    "firstName": "María",  
    "lastName": "García"  
  }  
}
```

5.8 GET /api/v1/profile sin token — 401

```
GET /api/v1/profile (sin header ni cookie)
```

```
{  
  "error": "No autenticado"  
}
```

5.9 GET /api/v1/admin con rol user — 403

```
GET /api/v1/admin  
Authorization: Bearer eyJhbGci... (token de usuario con role: 'user')
```

```
{  
  "error": "Acceso denegado"  
}
```

5.10 POST /api/products sin autenticación — 401

```
POST /api/products (sin token)
```

```
{  
  "error": "No autenticado"  
}
```

5.11 GET /api/products — ruta pública (sin auth)

```
GET /api/products?limit=2
```

```
{  
  "status": "success",  
  "payload": [  
    {  
      "_id": "69fc8182551ed564e3d96d19",  
      "title": "Casio G-Shock GA-2100",  
      "description": "Reloj deportivo ultrarresistente con caja octogonal...",  
      "price": 99,  
      "category": "Sport"  
    }  
  ]  
}
```



```
    }
  ],
  "totalPages": 5,
  "page": 1
}
```

5.12 Logout exitoso

POST /api/v1/auth/logout (con cookie de sesión)

```
{
  "message": "Logout exitoso"
}
```

5.13 Session después del logout — 401

GET /api/v1/session (con cookie connect.sid ya destruida)

```
{
  "error": "Sin sesión activa"
}
```

5.14 Suite de tests — 14/14 pasando

RUN v4.1.7

```
Test Files 3 passed (3)
  Tests 14 passed (14)
  Start at 10:20:58
  Duration 748ms
```

Detalle:

```
tests/managers/UserManager.test.js (7 tests)
✓ register → lanza EMAIL_TAKEN si el email ya existe
✓ register → hashea la contraseña antes de guardar
✓ register → devuelve el usuario creado
✓ findOrCreateGithub → retorna usuario existente si githubId ya está registrado
✓ findOrCreateGithub → crea usuario nuevo si githubId no existe
✓ findOrCreateGoogle → retorna usuario existente si googleId ya está registrado
✓ findOrCreateGoogle → crea usuario nuevo si googleId no existe
```

```
tests/middlewares/authJWT.test.js (4 tests)
✓ devuelve 401 si no hay token
✓ llama next() y adjunta req.user para token válido en cookie
✓ llama next() para token válido en Authorization header
✓ devuelve 401 para token inválido
```

```
tests/middlewares/authRole.test.js (3 tests)
✓ devuelve 401 si req.user no existe
✓ devuelve 403 si role es user
✓ llama next() si role es admin
```

6. Instrucciones de Instalación Local

6.1 Requisitos previos

- **Node.js** `>= 20.8.0`
- **MongoDB** corriendo en `localhost:27017` (o URI de Atlas en `.env`)
- **Git**
- Cuenta en **GitHub** y/o **Google Cloud** para configurar OAuth (opcional)

6.2 Dependencias del proyecto

```
{
  "dependencies": {
    "bcrypt": "^6.0.0",
    "connect-mongo": "^6.0.0",
    "cookie-parser": "^1.4.7",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-handlebars": "^7.1.3",
    "express-session": "^1.19.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^8.4.0",
    "mongoose-paginate-v2": "^1.8.3",
    "passport": "^0.7.0",
    "passport-github2": "^0.1.12",
    "passport-google-oauth2": "^0.2.0",
    "passport-local": "^1.0.0",
    "socket.io": "^4.7.5"
  }
}
```

6.3 Pasos para ejecutar localmente

```
# 1. Clonar el repositorio
git clone https://github.com/bautistacorte05/chronoshop-backend.git
cd chronoshop-backend

# 2. Instalar dependencias
npm install

# 3. Crear archivo de variables de entorno
cp .env.example .env
# Editar .env con tus valores (ver sección 6.4)

# 4. (Opcional) Cargar datos iniciales de productos
npm run seed

# 5. Iniciar el servidor
npm run dev      # desarrollo (hot reload)
# o
npm start       # producción

# 6. Ejecutar tests
npm test
```

El servidor queda disponible en `http://localhost:8080`.

6.4 Archivo `.env.example` comentado

Método	URL	Auth	Descripción
GET	/api/v1/profile	JWT	Perfil del usuario
GET	/api/v1/admin	JWT + Admin	Panel de administración
GET	/auth/github	—	Inicio OAuth GitHub
GET	/auth/google	—	Inicio OAuth Google
GET	/api/products	—	Lista de productos (pública)
POST	/api/products	JWT + Admin	Crear producto
PUT	/api/products/:pid	JWT + Admin	Actualizar producto
DELETE	/api/products/:pid	JWT + Admin	Eliminar producto
GET	/login	—	Vista de login (SSR)
GET	/register	—	Vista de registro (SSR)